```
                break;
            }
        }
        if (idx >= 0) {
            System.out.println("Το όνομα " + schElement + " βρέθηκε στην θέση
" + idx);
        } else {
            System.out.println("Το όνομα " + schElement + " δεν βρέθηκε");
        }
    }
```

Κώδικας 6.13 *Σειριακή αναζήτηση σε πίνακα αλφαριθμητικών σειρών*

6.4.2 Μέγιστο ή ελάχιστο στοιχείο

Ένα άλλο πρόβλημα είναι η αναζήτηση του μέγιστου στοιχείου ενός πίνακα. Πρόκειται για απλή επεξεργασία. Παραθέτουμε τον κώδικα υλοποίησης και στη συνέχεια τον σχολιάζουμε:

```
    public static void max() { //Κώδικας 6.14
        int[] t = {2, 6, 1, 9, 15};
        int max = t[0];
        int idx = 0;
        for (int i = 1; i < t.length; i++) {
            if (t[i] > max) {
                max = t[i];
                idx = i;
            }
        }
        System.out.println("Το μεγαλύτερο στοιχείο είναι το " + max + " και
βρίσκεται στην θέση " + idx);
```

Κώδικας 6.14 *Εύρεση του μεγαλύτερου στοιχείου μονοδιάστατου πίνακα*

Στον κώδικα 6.14 καταρχάς ορίζουμε τον πίνακα t. Στη συνέχεια, δύο βοηθητικές μεταβλητές, την max και την idx. Η max αρχικοποιείται στην τιμή του στοιχείου στη θέση μηδέν του πίνακα. Επίσης, η idx αρχικοποιείται ώστε να δείχνει στη θέση μηδέν. Ακολούθως, προσπελαύνουμε τα στοιχεία του πίνακα ένα προς ένα και όποτε βρίσκουμε τιμή μεγαλύτερη από την τρέχουσα τιμή του max ενημερώνουμε την max και την idx. Στο τέλος της διαδικασίας, η max έχει τη μεγαλύτερη τιμή του πίνακα και η idx τη θέση της.

Προσέξτε πως ο βρόχος for ξεκινάει από 1 και όχι από 0. Πράγματι, στο συγκεκριμένο πρόβλημα δεν υπάρχει λόγος να ξεκινήσει η σύγκριση από το 0. Αντίθετα, αν ξεκινούσε από το 0, κατά το πρώτο επαναληπτικό βήμα θα συγκρινόταν η max με το στοιχείο t[0]. Η max όμως σε αυτό το βήμα έχει τιμή ίση με το t[0], οπότε είναι περιττή η σύγκριση.

Στην περίπτωση που ο πίνακας έχει δύο στοιχεία ίσα μεταξύ τους και μεγαλύτερα από τα υπόλοιπα στοιχεία του πίνακα, ο κώδικας 6.14 εντοπίζει το στοιχείο στη μικρότερη θέση. Αν επιθυμούμε να βρούμε το στοιχείο στη μεγαλύτερη θέση αρκεί να τροποποιήσουμε τη συνθήκη της if σε t[i]>=max.

Αντίστοιχα, η εύρεση του μικρότερου στοιχείου, βρίσκεται αν αντικαταστήσουμε τη συνθήκη ελέγχου της if με την έκφραση t[i]<min ή t[i]<=min.

6.4.3 Άθροισμα

Ένα άλλο τυπικό πρόβλημα είναι ο υπολογισμός του αθροίσματος των στοιχείων ενός πίνακα. Πρόκειται επίσης για μια σειριακή προσπέλαση των στοιχείων του πίνακα και προσαύξηση ενός αθροιστή κατά την τιμή κάθε στοιχείου. Ο κώδικας 6.15 παρουσιάζει τη λύση.

```
    public static void sum() { //Κώδικας 6.15
        int[] t = {2, 6, 1, 9, 15};
```

```
        int sum = 0;
        for (int i = 0; i < t.length; i++) {
            sum += t[i];
        }
        System.out.println("Το άθροισμα είναι " + sum);
    }
```

Κώδικας 6.15 *Άθροισμα στοιχείων μονοδιάστατου πίνακα*

6.4.4 Ανακατανομή

Σε πολλές εφαρμογές απαιτείται τυχαία ανακατανομή των στοιχείων ενός πίνακα. Για κάθε στοιχείο του πίνακα, υπολογίζουμε μια τυχαία θέση μέσα στα όρια του πίνακα και αντιμεταθέτουμε το στοιχείο στην τρέχουσα θέση με το στοιχείο στην τυχαία θέση.

```
    public static void shuffle() { //Κώδικας 6.16
        int[] t = {2, 6, 1, 9, 15};
        Arrays.sort(t);
        System.out.println(Arrays.toString(t));
        Random r = new Random();

        for (int i = 0; i < t.length; i++) {
            int rIdx = r.nextInt(t.length);
            int tmp = t[i];
            t[i] = t[rIdx];
            t[rIdx] = tmp;
        }
        System.out.println(Arrays.toString(t));
    }
```

Κώδικας 6.16 *Ανακατανομή στοιχείων πίνακα*

Στον κώδικα 6.16 δημιουργούμε το πίνακα t, τον ταξινομούμε και τον τυπώνουμε έτσι ώστε να είναι εμφανής η διαφορά από τον ανακατεμένο πίνακα. Στη συνέχεια υλοποιείται ο αλγόριθμος ανακατανομής που περιγράφηκε αμέσως προηγουμένως.

6.4.5 Αρχικοποίηση με τυχαίες τιμές

Πολλές φορές χρειαζόμαστε να γεμίσουμε έναν πίνακα με τυχαίες τιμές είτε για λόγους ελέγχου είτε επειδή η εφαρμογή μας το απαιτεί. Στον κώδικα 6.17 ο πίνακας γεμίζει με τυχαίες τιμές από το 0 έως 99.

```
    public static void fillRandom() { //Κώδικας 6.17
        int[] t = new int[20];
        Random r = new Random();
        for (int i = 0; i < t.length; i++) {
            t[i] = r.nextInt(100);
        }
        System.out.println(Arrays.toString(t));
    }
```

Κώδικας 6.17 *Αρχικοποίηση πίνακα με τυχαίες τιμές*

6.4.6 Τελικοί πίνακες

Η δεσμευμένη λέξη final μπορεί να χρησιμοποιηθεί και με μεταβλητές που αναφέρονται σε πίνακες. Η σημασία της είναι ακριβώς ίδια με τη σημασία που έχει όταν χρησιμοποιείται με μεταβλητές τιμής. Ωστόσο, επειδή οι μεταβλητές αναφοράς έχουν διαφορετική συμπεριφορά, συχνά παρατηρείται σύγχυση στον αρχάριο προγραμματιστή σε σχέση με τις επιπτώσεις του χαρακτηρισμού ως final ενός πίνακα.

Όπως αναφέρουμε στην ενότητα 3.1.2, ο χαρακτηρισμός ως final μιας μεταβλητής έχει ως συνέπεια να επιτρέπεται εκχώρηση στη μεταβλητή μόνο μία φορά. Το ίδιο ακριβώς συμβαίνει και με τις μεταβλητές πινάκων. Επομένως, ο κώδικας που ακολουθεί παράγει λάθος χρόνου μεταγλώττισης.

```
final int[] t={1,3,5,7};
t=new int[9];
```

Ωστόσο, η μεταβολή των στοιχείων του πίνακα είναι διαφορετική υπόθεση. Αυτή μπορεί να γίνει χωρίς να μεταβληθεί η αναφορά στον πίνακα.

Επομένως, ο ακόλουθος κώδικας είναι απολύτως σωστός:

```
final int[] t={1,3,5,7};
t[0]=10;
```

6.5 Λυμένες Ασκήσεις

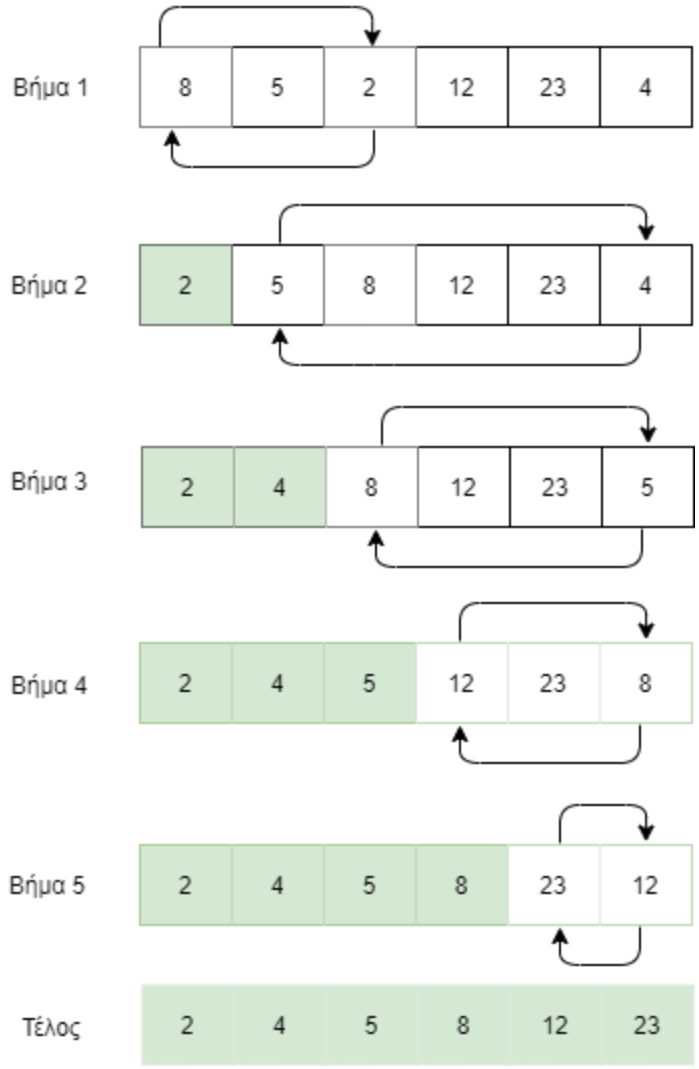
Παρουσιάζουμε εδώ μια σειρά από βασικές επεξεργασίες πινάκων.

6.5.1 Ταξινόμηση

Να αναπτύξετε συνάρτηση στην οποία να ορίσετε έναν πίνακα ακέραιων και στη συνέχεια να τον ταξινομήσετε κατ' αύξουσα σειρά χωρίς να χρησιμοποιήσετε συνάρτηση sort της Arrays.

Λύση

Υπάρχουν πολλοί αλγόριθμοι ταξινόμησης δεδομένων. Για τη λύση αυτής της άσκησης θα χρησιμοποιήσουμε ίσως τον απλούστερο όλων, γνωστό ως selection sort [1]. Το βασικό βήμα σε αυτήν την ταξινόμηση είναι η ανταλλαγή θέσης μεταξύ μικρότερου στοιχείου και του στοιχείου που είναι στην αρχή του αταξινόμητου μέρους του πίνακα. Το βήμα επαναλαμβάνεται από το πρώτο μέχρι το προτελευταίο στοιχείο του πίνακα. Με το τέλος της επανάληψης ο πίνακας είναι ταξινομημένος. Εξηγούμε λεπτομερέστερα με τη βοήθεια του σχήματος 6.2.



Σχήμα 6.2 *Αναπαράσταση του selection sort*

Βήμα 1: Ολόκληρος ο πίνακας είναι αταξινόμητος. Το πρώτο στοιχείο του αταξινόμητου πίνακα βρίσκεται στη θέση 0 και έχει τιμή 8. Το μικρότερο στοιχείο του πίνακα βρίσκεται στη θέση 2 και έχει τιμή 2. Τα στοιχεία θα ανταλλάξουν θέσεις.

Βήμα 2: Μετά την ανταλλαγή θέσεων του βήματος 1, το μικρότερο στοιχείο βρίσκεται στη θέση 0. Το τμήμα του πίνακα που περιλαμβάνει τη θέση 0 θεωρείται ταξινομημένο. Το μη ταξινομημένο τμήμα του πίνακα εκτείνεται από τη θέση 1 έως το τέλος του πίνακα. Επαναλαμβάνεται η λογική του βήματος 1 για το αταξινόμητο τμήμα του πίνακα. Το μικρότερο στοιχείο με τιμή 4 θα ανταλλάξει θέση με το στοιχείο με τιμή 5.

Βήμα 3: Με την ίδια λογική το 5 θα ανταλλάξει θέση με το 8.

Βήμα 4: Το 8 θα ανταλλάξει θέση με το 12

Βήμα 5: Το 12 θα ανταλλάξει θέση με το 23

Τέλος: Ο πίνακας είναι ταξινομημένος

Στην συνέχεια δίνουμε την υλοποίηση της selectionSort:

```
1   static void selectionSort(String[] args) { //Κώδικας 6.18
2       int[] tbl = {2, 1, 6, 3, 4, 9, 8, 5, 7, 4};
3       for (int i = 0; i < tbl.length - 1; i++) {
4           int min = tbl[i];
5           int idx = i;
6           for (int j = i + 1; j < tbl.length; j++) {
7               if (tbl[j] < min) {
8                   min = tbl[j];
9                   idx = j;
10              }
11          }
12
13          if (idx != i) {
14             int tmp = tbl[i];
15             tbl[i] = tbl[idx];
16             tbl[idx] = tmp;
17         }
18      }
19      System.out.println(Arrays.toString(tbl));
20  }
```

Κώδικας 6.18 *Υλοποίηση του selection sort*

Στη γραμμή 2 ορίζουμε έναν πίνακα ακέραιων. Στη γραμμή 3 ξεκινάμε τη βασική επαναληπτική διαδικασία. Στις γραμμές 4 έως 11 αναζητούμε το μικρότερο στοιχείο του πίνακα από τη θέση i και μετά. Αν το μικρότερο στοιχείο βρίσκεται σε θέση διαφορετική από το i, αντιμεταθέτουμε τα δύο στοιχεία (γραμμές 11 έως 15). Ο εξωτερικός βρόχος (γραμμή 3) ελέγχει μέχρι tbl.length-2. Αυτό είναι λογικό, καθώς το tbl.length-1 είναι η τελευταία θέση του πίνακα και δεν έχει νόημα να ελέγξουμε αν μετά από αυτήν υπάρχει στοιχείο με μικρότερη τιμή. Στη γραμμή 19 εμφανίζουμε τον ταξινομημένο πίνακα.

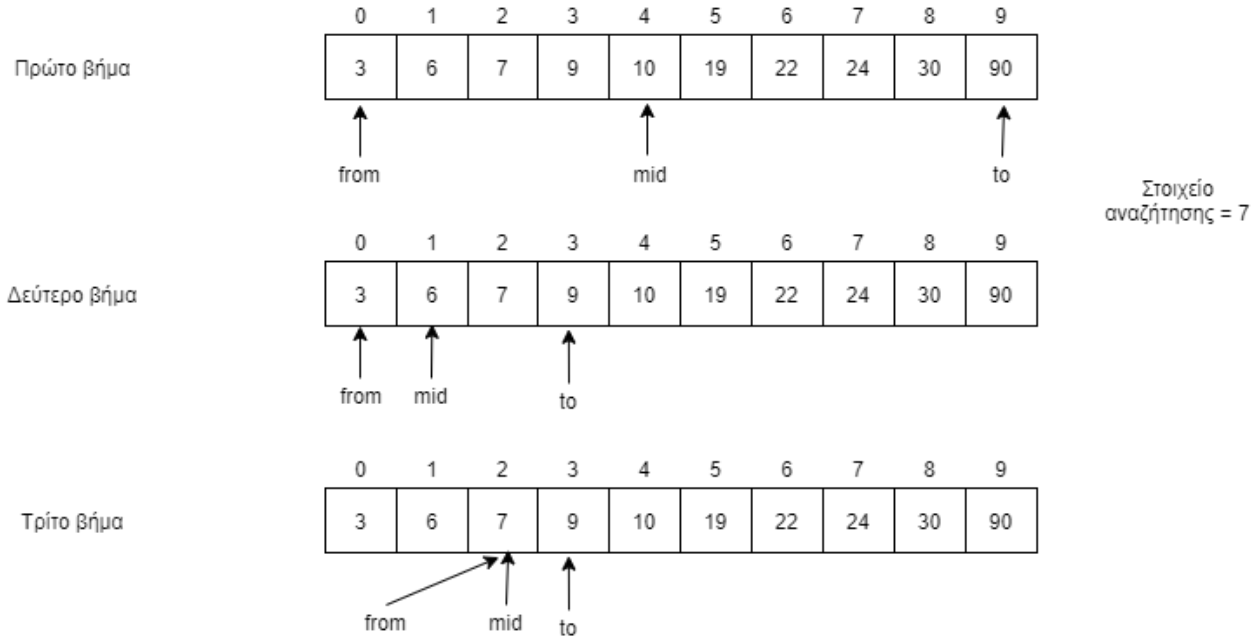
6.5.2 Δυαδική αναζήτηση

Να αναπτύξετε συνάρτηση Java στην οποία να ορίσετε έναν μονοδιάστατο πίνακα ακέραιων, έναν ακέραιο και στη συνέχεια να πραγματοποιήσετε δυαδική αναζήτηση στον πίνακα και να βρείτε τον ακέραιο χωρίς να χρησιμοποιήσετε τις συναρτήσεις δυαδικής αναζήτησης της Arrays.

Λύση

Η βασική ιδέα της δυαδικής αναζήτησης (binary search) [2] είναι να εξετάζουμε κάθε φορά ένα τμήμα του πίνακα για το οποίο ελέγχουμε αν η τιμή που αναζητούμε είναι μεγαλύτερη, μικρότερη ή ίση από την τιμή στο μέσο του τμήματος. Αν είναι ίση, το στοιχείο αναζήτησης βρέθηκε. Αν είναι μικρότερη, μας μένει να ψάξουμε μόνο στο πρώτο μισό μέρος του τμήματος του πίνακα. Αν είναι μεγαλύτερη, μας μένει να ψάξουμε μόνο στο δεύτερο μισό μέρος του τμήματος του πίνακα. Με αυτόν τον τρόπο, η δυαδική αναζήτηση υποδιπλασιάζει σε κάθε επαναληπτικό βήμα τα στοιχεία του πίνακα που μένουν προς έλεγχο. Κατά το πρώτο βήμα, το τμήμα του πίνακα που έχουμε να ψάξουμε συμπίπτει με ολόκληρο τον πίνακα.

Για να υλοποιήσουμε τη δυαδική αναζήτηση χρειαζόμαστε πέραν του πίνακα και του στοιχείου αναζήτησης και δύο δείκτες, ας τους ονομάσουμε from και to, που καθορίζουν τα όρια του τμήματος που ελέγχουμε σε κάθε επαναληπτικό βήμα. Στο σχήμα 6.3 παρουσιάζεται παράδειγμα δυαδικής αναζήτησης:



Σχήμα 6.3 *Παράδειγμα δυαδικής αναζήτησης*

Στην αρχή της αναζήτησης, ο δείκτης from δείχνει στην αρχή του πίνακα και ο δείκτης to στο τέλος. Συμπεριλαμβάνεται έτσι όλος ο πίνακας στην αναζήτηση. Με βάση τις τιμές των from και to υπολογίζεται το μέσο, mid, του πίνακα. Στη συνέχεια, εφόσον το στοιχείο αναζήτησης είναι μεγαλύτερο από την τιμή στο μέσο του πίνακα και ο πίνακας είναι ταξινομημένος κατ' αύξουσα σειρά, εξάγεται το συμπέρασμα πως το στοιχείο αναζήτησης, αν υπάρχει, θα πρέπει να αναζητηθεί στο δεύτερο τμήμα του πίνακα, δηλαδή στο τμήμα που αρχίζει από mid+1 και το τέλος του συμπίπτει με το τέλος του πίνακα. Αντίθετα, αν όπως στο παράδειγμά μας, το στοιχείο αναζήτησης είναι μικρότερο από την τιμή στο μέσο του πίνακα, τότε η αναζήτηση θα πρέπει να συνεχίσει με το πρώτο μισό τμήμα του πίνακα, δηλαδή να εφαρμοστεί στο τμήμα from έως mid-1.

Έτσι στο δεύτερο βήμα, ο δείκτης to γίνεται ίσος με mid-1 και άρα το νέο μέσο τοποθετείται στη θέση 1 του πίνακα και δείχνει μια θέση μετά το μέσο του πίνακα. Στη συνέχεια, το στοιχείο αναζήτησης είναι μεγαλύτερο από την τιμή στη θέση 1, οπότε ο δείκτης from γίνεται mid+1. Το νέο μέσο βρίσκεται στη θέση 2 όπου βρίσκεται και το στοιχείο αναζήτησης. Τέλος, ελέγχεται το μέσο και βρίσκεται ίσο με το στοιχείο αναζήτησης, οπότε η αναζήτηση διακόπτεται.

Ο κώδικας 6.19 παρέχει την υλοποίηση σε Java του αλγόριθμου δυαδικής αναζήτησης.

```
public static void binarySearch() { //Κώδικας 6.19
    int[] tbl = new int[]{1, 3, 4, 5, 7, 9, 11, 23, 34, 50};
    int idx = -1;
    final int schElement = 24;
    int from = 0, to = tbl.length - 1, mid = (to - from) / 2;
    while (to >= from) {
        if (tbl[mid] == schElement) {
            idx = mid;
            break;
        } else if (schElement < tbl[mid]) {
            to = mid - 1;
        } else {
            from = mid + 1;
        }
        mid = from + (to - from) / 2;
```

```
        }
        if (idx == -1) {
            System.out.println("Element " + schElement + " was not found");
        } else {
            System.out.println("Element " + schElement + " found at position
" + idx);
        }
    }
```

Κώδικας 6.19 *Υλοποίηση δυαδικής αναζήτησης*

Στον κώδικα 6.19 καθορίζουμε καταρχάς τον πίνακα στον οποίο θα εφαρμόσουμε τη δυαδική αναζήτηση. Στη συνέχεια ορίζουμε τις απαραιτητες βοηθητικές μεταβλητές idx, from, to και mid. Η μεταβλητή from αρχικοποιείται στο 0 και η to στο tbl.length-1. Έτσι, η αναζήτηση αρχικά αφορά όλο τον πίνακα. Αμέσως μετά, καθορίζουμε το στοιχείο αναζήτησης, schElement. Στη συνέχεια, η ροή εισέρχεται σε έναν βρόχο while που ελέγχεται από τη συνθήκη to>=from. Όσο αυτή η συνθήκη είναι αληθής, υπάρχει τμήμα του πίνακα που δεν έχει ελεγχθεί. Στη συνέχεια, αν η τιμή στο μέσο του πίνακα είναι ίση με την τιμή που αναζητούμε, το στοιχείο βρέθηκε, οπότε ενημερώνεται η idx και διακόπτεται η while, αλλιώς, αν το στοιχείο αναζήτησης είναι μικρότερο από την τιμή στο μέσο του πίνακα, ενημερώνεται η μεταβλητή to ώστε η αναζήτηση να περιοριστεί στο πρώτο μισό του πίνακα. Αν όμως το στοιχείο αναζήτησης είναι μεγαλύτερο από την τιμή στο μέσο του πίνακα, ενημερώνεται η from ώστε η αναζήτηση να συνεχιστεί στο δεύτερο μισό του πίνακα. Στο επόμενο βήμα υπολογίζεται το νέο μέσο. Τέλος, έξω από τον βρόχο while ενημερώνουμε με τα αποτελέσματα της αναζήτησης με την εμφάνιση κατάλληλων μηνυμάτων.

6.5.3 Ισότητα Πινάκων

Να αναπτύξετε συνάρτηση στην οποία να ορίσετε τους ακόλουθους πίνακες:

```
String[] tbl1 = new String[]{new String("John"), "Maria",
"Kely"};
String[] tbl2 = {"John", "Kely", "Maria"};
```

Στη συνέχεια να ελέγξετε αν είναι ίσοι μεταξύ τους με βάση τον ακόλουθο ορισμό ισότητας: Δύο πίνακες από Strings είναι ίσοι μεταξύ τους, εφόσον έχουν το ίδιο μέγεθος και κάθε στοιχείο του ενός είναι και στοιχείο του άλλου ανεξάρτητα από τη θέση που έχουν τα στοιχεία στους πίνακες.
Προσέξτε πως αυτός ο ορισμός ισότητας πινάκων διαφέρει από τον ορισμό στον οποίο βασίζονται οι συναρτήσεις equals της Array.

Λύση

```
public static void stringEquality() { //Κώδικας 6.20
    String[] tbl1 = new String[]{new String("John"), "Maria", "Kely"};
    String[] tbl2 = {"John", "Kely", "Maria"};
    boolean equal = tbl1.length == tbl2.length;
    for (int i = 0; i < tbl1.length && equal; i++) {
        boolean found = false;
        for (String cName : tbl2) {
            if (tbl1[i].equals(cName)) {
                found = true;
                break;
            }
        }
        equal = found;
    }
    System.out.println(equal);
```

```
}
```

Κώδικας 6.20 *Έλεγχος ισότητας πινάκων*

Στον κώδικα 6.20 ορίζουμε αρχικά τους πίνακες όπως δίνονται στην εκφώνηση. Στη συνέχεια αρχικοποιούμε τη μεταβλητή equal στην τιμή της έκφρασης tbl1.length == tbl2.length. Αν οι πίνακες δεν έχουν ίδιο μήκος, η equal γίνεται false. Στη συνέχεια η ροή δεν μπαίνει στον βρόχο for και απλά εμφανίζει την τιμή false. Αν όμως τα μήκη των πινάκων είναι ίσα μεταξύ τους, τότε η ροή εισέρχεται στον εξωτερικό βρόχο for. Εκεί ελέγχεται κάθε στοιχείο του tbl1 αν υπάρχει και στον tbl2. Όσο ένα στοιχείο του tbl1 βρίσκεται και στον tbl2, η βοηθητική μεταβλητή found γίνεται true και αντίστοιχα ενημερώνεται και η equal. Αν ένα στοιχείο του tbl1 δεν βρεθεί στον tbl2, η found γίνεται false και διακόπτεται η εσωτερική for. Ακολούθως γίνεται false η equal η οποία θα διακόψει και την εξωτερική for.

6.5.4 Αντιστροφή πίνακα

Να γράψετε κώδικα ο οποίος με δεδομένο τον πίνακα int[] tbl = {1, 2, 3, 14, 5, 6, 7, 8, 9} παράγει έναν πίνακα που περιέχει τα ίδια στοιχεία με τον tbl σε αντεστραμμένη διάταξη, δηλαδή στη θέση 0 έχει το 9 και στη θέση 8 το 1.

Λύση

```
public static void reverse() { //Κώδικας 6.21
    int[] tbl = {1, 2, 3, 14, 5, 6, 7, 8, 9};
    int[] reversed = new int[tbl.length];
    for (int i = 0; i < tbl.length; i++) {
        reversed[tbl.length - 1 - i] = tbl[i];
    }
    System.out.print(Arrays.toString(reversed));
}
```

Κώδικας 6.21 *Παραγωγή πίνακα με αντεστραμμένη διάταξη στοιχείων*

Καταρχάς δημιουργούμε τον πίνακα reversed με ίδιο μήκος με τον tbl. Στη συνέχεια τοποθετούμε στη θέση tbl.length - 1 – i του reversed το στοιχείο που βρίσκεται στη θέση i του tbl.

6.5.5 Συνένωση πινάκων

Να γράψετε κώδικα που με δεδομένους τους πίνακες int[] t1={1,2,3,4,5}, t2={6,7,8,9} παράγει έναν πίνακα που συνενώνει τους t1 και t2.

Λύση

```
public static void merge() { //Κώδικας 6.22
    int[] t1 = {1, 2, 3, 4, 5}, t2 = {6, 7, 8, 9};
    int[] rVal = new int[t1.length + t2.length];
    System.arraycopy(t1, 0, rVal, 0, t1.length);
    System.arraycopy(t2, 0, rVal, t1.length, t2.length);
    System.out.println(Arrays.toString(rVal));
}
```

Κώδικας 6.22 *Συνένωση πινάκων*

Αφού δηλώσαμε τους πίνακες t1 και t2, δημιουργούμε τον πίνακα rVal με μήκος ικανό να φιλοξενήσει τα δεδομένα των t1 και t2. Στη συνέχεια, με την arraycopy μεταφέρουμε τα δεδομένα από τους t1 και t2. Τέλος, τυπώνουμε τον rVal ώστε να δείξουμε πως η συνένωση έγινε σωστά.